



Python Programming Fundamentals

The Product Class

Programming Fundamentals Team

SETU

2026-06-12



Outline

1. Designing the Product Class
2. The `__init__` Method
3. Getters and Setters
4. The `__str__` Method
5. Creating Product Instances
6. Full Product Class
7. UML Class Diagram



Outline

1. Designing the Product Class

2. The `__init__` Method

3. Getters and Setters

4. The `__str__` Method

5. Creating Product Instances

6. Full Product Class

7. UML Class Diagram



1.1 What does a Product need?

Every product in our shop has:

- **name** — a string (e.g. "Apple")
- **price** — a float (e.g. 1.99)
- **product_id** — an integer (e.g. 101)

We model this as a class so we can create many product objects from one blueprint.

```
class Product:
    def __init__(self, name, price, product_id):
        self.__name = name
        self.__price = price
        self.__product_id = product_id
```



Outline

1. Designing the Product Class
- 2. The `__init__` Method**
3. Getters and Setters
4. The `__str__` Method
5. Creating Product Instances
6. Full Product Class
7. UML Class Diagram



2.1 Initialising a Product

`__init__` is the **constructor** — it runs automatically when you create a new object.

```
class Product:
    def __init__(self, name: str, price: float, product_id: int):
        self.__name = name
        self.__price = price
        self.__product_id = product_id
```

- The double underscore prefix (`__`) makes attributes **private**
- Private attributes cannot be accessed directly from outside the class
- This is **encapsulation** — hiding internal data



Outline

1. Designing the Product Class
2. The `__init__` Method
- 3. Getters and Setters**
4. The `__str__` Method
5. Creating Product Instances
6. Full Product Class
7. UML Class Diagram



3.1 Accessing Private Attributes

```
class Product:
    # ... __init__ as above ...

    def get_name(self) -> str:
        return self.__name

    def set_name(self, name: str):
        self.__name = name

    def get_price(self) -> float:
        return self.__price

    def set_price(self, price: float):
        self.__price = price
```



3.1 Accessing Private Attributes

```
def get_product_id(self) -> int:  
    return self.__product_id
```

Getters **return** the value. Setters **update** the value (and can validate it).



Outline

1. Designing the Product Class
2. The `__init__` Method
3. Getters and Setters
- 4. The `__str__` Method**
5. Creating Product Instances
6. Full Product Class
7. UML Class Diagram



4.1 Human-readable representation

```
class Product:
    # ... previous code ...

    def __str__(self) -> str:
        return (f"Product [{self.__product_id}]: "
                f"{self.__name} – €{self.__price:.2f}")
```

`__str__` is a **dunder method** (double underscore). Python calls it automatically when you:

- use `print(product)`
- use `str(product)`
- embed the object in an f-string: `f"{product}"`



Outline

1. Designing the Product Class
2. The `__init__` Method
3. Getters and Setters
4. The `__str__` Method
- 5. Creating Product Instances**
6. Full Product Class
7. UML Class Diagram



5.1 Using the class

```
# Create product objects
apple = Product("Apple", 1.99, 101)
bread = Product("Bread", 2.50, 102)
milk  = Product("Milk", 1.20, 103)

# Print them – calls __str__
print(apple)
print(bread)
print(milk)
```

Output:

```
Product [101]: Apple – €1.99
Product [102]: Bread – €2.50
Product [103]: Milk – €1.20
```



Outline

1. Designing the Product Class
2. The `__init__` Method
3. Getters and Setters
4. The `__str__` Method
5. Creating Product Instances
- 6. Full Product Class**
7. UML Class Diagram



6. Full Product Class

```
class Product:
    def __init__(self, name: str, price: float, product_id: int):
        self.__name = name
        self.__price = price
        self.__product_id = product_id

    def get_name(self) -> str:
        return self.__name
    def set_name(self, name: str):
        self.__name = name

    def get_price(self) -> float:
        return self.__price
    def set_price(self, price: float):
        self.__price = price
```



6. Full Product Class

```
def get_product_id(self) -> int:
    return self.__product_id

def __str__(self) -> str:
    return (f"Product [{self.__product_id}]: "
            f"{self.__name} – €{self.__price:.2f}")
```



Outline

1. Designing the Product Class
2. The `__init__` Method
3. Getters and Setters
4. The `__str__` Method
5. Creating Product Instances
6. Full Product Class
- 7. UML Class Diagram**



7.1 Product

Product

- `__name__: str`
 - `__price__: float`
 - `__product_id__: int`

1. `get_name() : str`
 - + `set_name(name: str)`
 - + `get_price() : float`
 - + `set_price(price: float)`
 - + `get_product_id() : int`
 - + `__str__() : str`



Thanks for Watching - Any questions?

